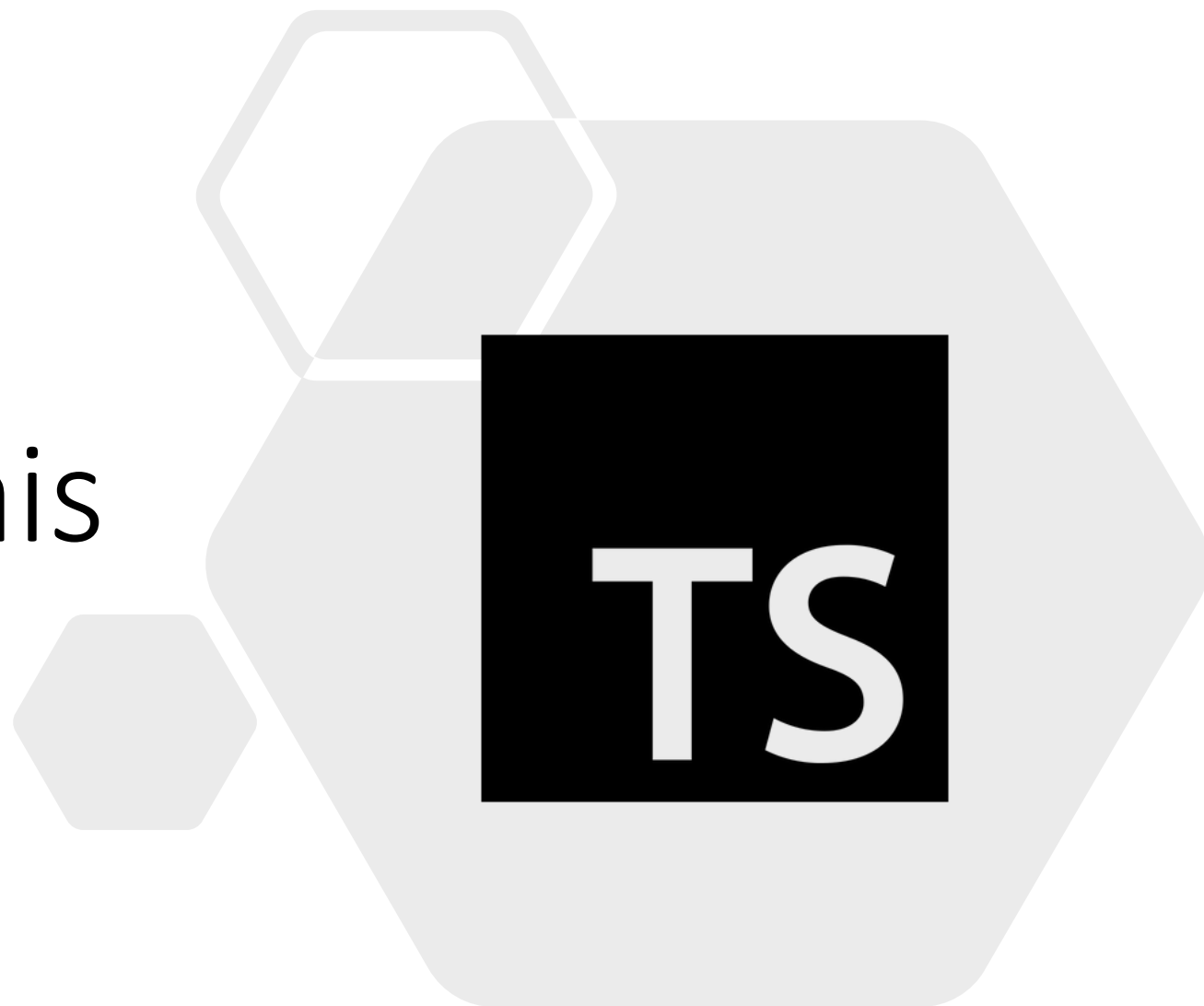
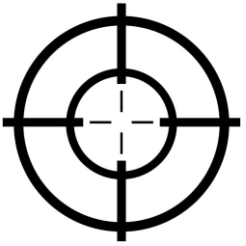


Padrões comportamentais



Padrões comportamentais



Na engenharia de software, os padrões de projeto comportamentais auxiliam na comunicação entre objetos, de modo a aumentar sua flexibilização.

O objetivo destes padrões é garantir a interação entre os objetos, de tal forma que eles possam se comunicar facilmente entre si e com baixo acoplamento. Isso significa que a implementação e o cliente (quem consome o código ou sua execução) devem ser fracamente acoplados para evitar codificação excessiva e dependências desnecessárias.

Strategy

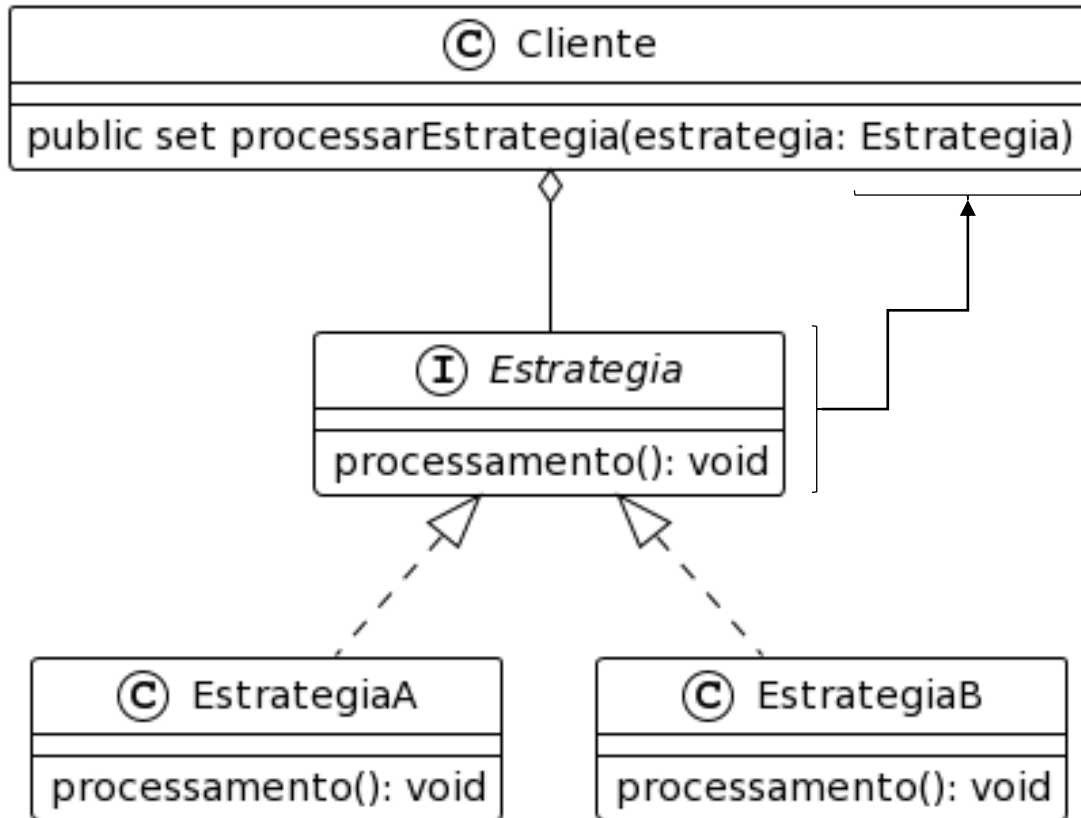
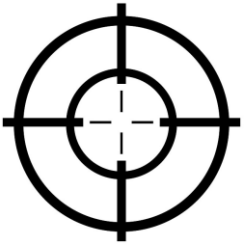
A “estratégia” (strategy) é um padrão de projeto comportamental, que permite definir uma família de algoritmos, encapsular cada um deles e torná-los intercambiáveis.

O principal resultado da aplicação deste padrão é a melhoria da manutenção do código. A manutenção de código é algo inerente ao desenvolvimento e evolução de qualquer aplicação.

Strategy

O padrão ocorre a partir da definição de um conjunto de classes, que possam ser alteradas em tempo de execução, ou seja, sem a necessidade de “parar” a execução do software. Os objetos destas classes podem trabalhar de forma independente, ocupando o mesmo espaço de execução liberado por uma interface comum a todos.

Strategy UML



O código (objeto ou sistema) cliente consome uma estratégia. Estratégia é uma interface e, portanto, não possui execução, mas permite implementações que podem ser executadas no seu lugar, através do polimorfismo.

Implementação Strategy



```
export default interface Estrategia {  
  processamento(): void  
}  
  
export default class EstrategiaA implements Estrategia {  
  processamento(): void {  
    console.log(`Objeto do tipo estrategia A em execução`);  
  }  
}  
  
export default class EstrategiaB implements Estrategia {  
  processamento(): void {  
    console.log(`Objeto do tipo estrategia B em execução`);  
  }  
}
```

A diagram illustrating the Strategy pattern implementation. It shows three code blocks. The first block is an interface 'Estrategia' with a method 'processamento(): void'. The second block is a class 'EstrategiaA' that implements 'Estrategia', with its own 'processamento()' method that logs 'Objeto do tipo estrategia A em execução'. The third block is a class 'EstrategiaB' that also implements 'Estrategia', with its own 'processamento()' method that logs 'Objeto do tipo estrategia B em execução'. Arrows indicate that both 'EstrategiaA' and 'EstrategiaB' implement the 'Estrategia' interface.

As classes A e B são estratégias, algoritmos intercambiáveis que podem “ocupar” o lugar da interface durante a execução do código.

Implementação Strategy

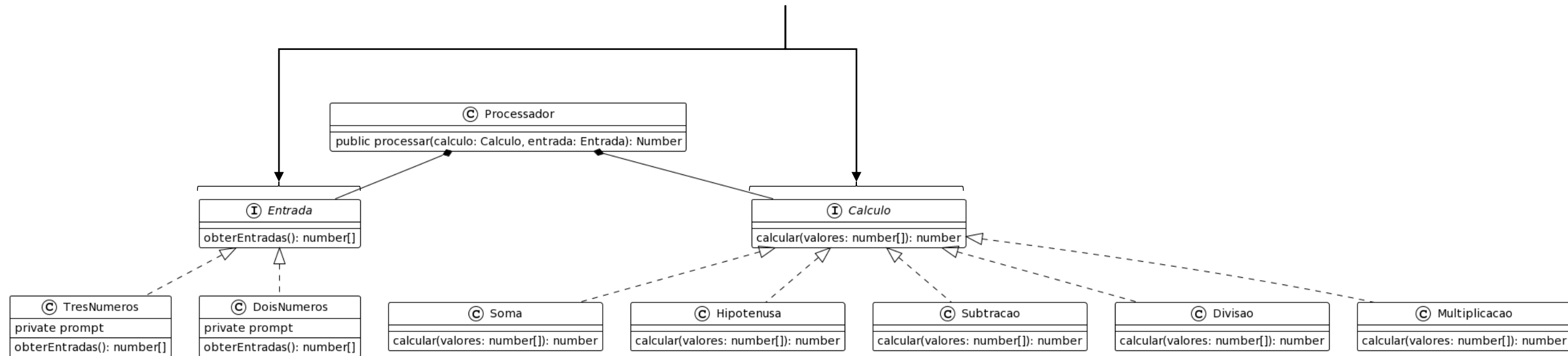


```
export default interface Estrategia {  
    processamento(): void  
}  
  
export default class Cliente {  
    public set processarEstrategia(estrategia: Estrategia) {  
        estrategia.processamento()  
    }  
}
```

A diagram illustrating the Strategy design pattern. It shows an interface named 'Estrategia' with a method 'processamento(): void'. Below it, a class named 'Cliente' is shown with a method 'processarEstrategia(estrategia: Estrategia)'. A line connects the 'Estrategia' interface to the 'processarEstrategia' method in the 'Cliente' class, indicating that the class implements the interface.

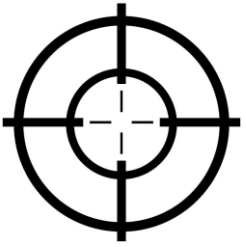
O código “cliente” consome estratégias. As estratégias são algoritmos (objetos) que podem variar seu comportamento, mas respeitam a interface que estabelece seu funcionamento.

A classe “processador” define um método denominado de “processar”, que depende das interfaces “entrada” e “cálculo”, mas não depende de um tipo específico de cálculo ou entrada.



Neste exemplo, a estrutura do código está flexível e com baixo acoplamento. Isto permite evoluções futuras em que um novo tipo de entrada ou cálculo pode ser adicionado facilmente, sem “quebrar” o funcionamento do código e com pouquíssima modificação na estrutura.

Strategy UML



```
export default interface Entrada {  
  obterEntradas(): number[]  
}
```

```
export default class DoisNumeros implements Entrada {  
  private prompt = promptSync()  
  obterEntradas(): number[] {  
    let numero1 = this.prompt(`Por favor, insira o numero 1: `)  
    let numero2 = this.prompt(`Por favor, insira o numero 2: `)  
    let numeros = [Number.parseFloat(numero1), Number.parseFloat(numero2)]  
    return numeros  
  }  
}
```

```
export default interface Calculo {  
  calcular(valores: number[]): number  
}
```

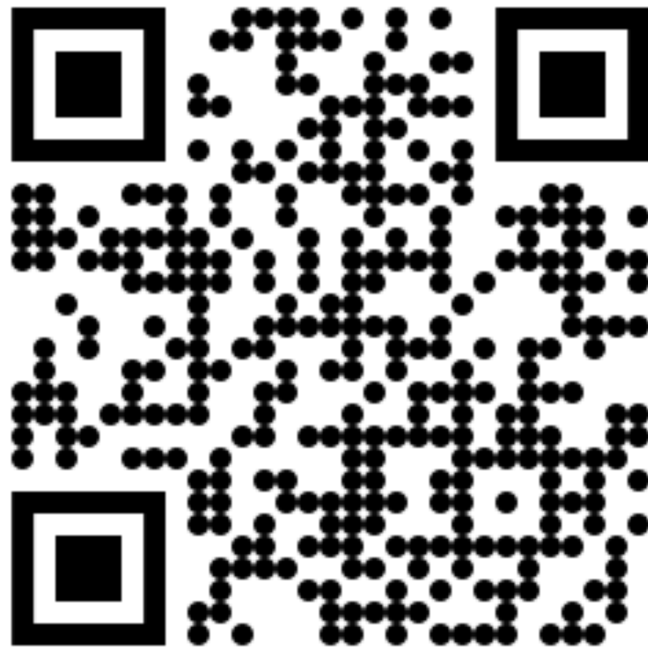
```
export default class Soma implements Calculo {  
  calcular(valores: number[]): number {  
    let soma = 0  
    valores.forEach(valor => {  
      soma = soma + valor  
    })  
    return soma  
  }  
}
```

Implementação Strategy



<https://github.com/gerson-pn>

`/strategy-typeScript`



`/strategy-typeScript-example`



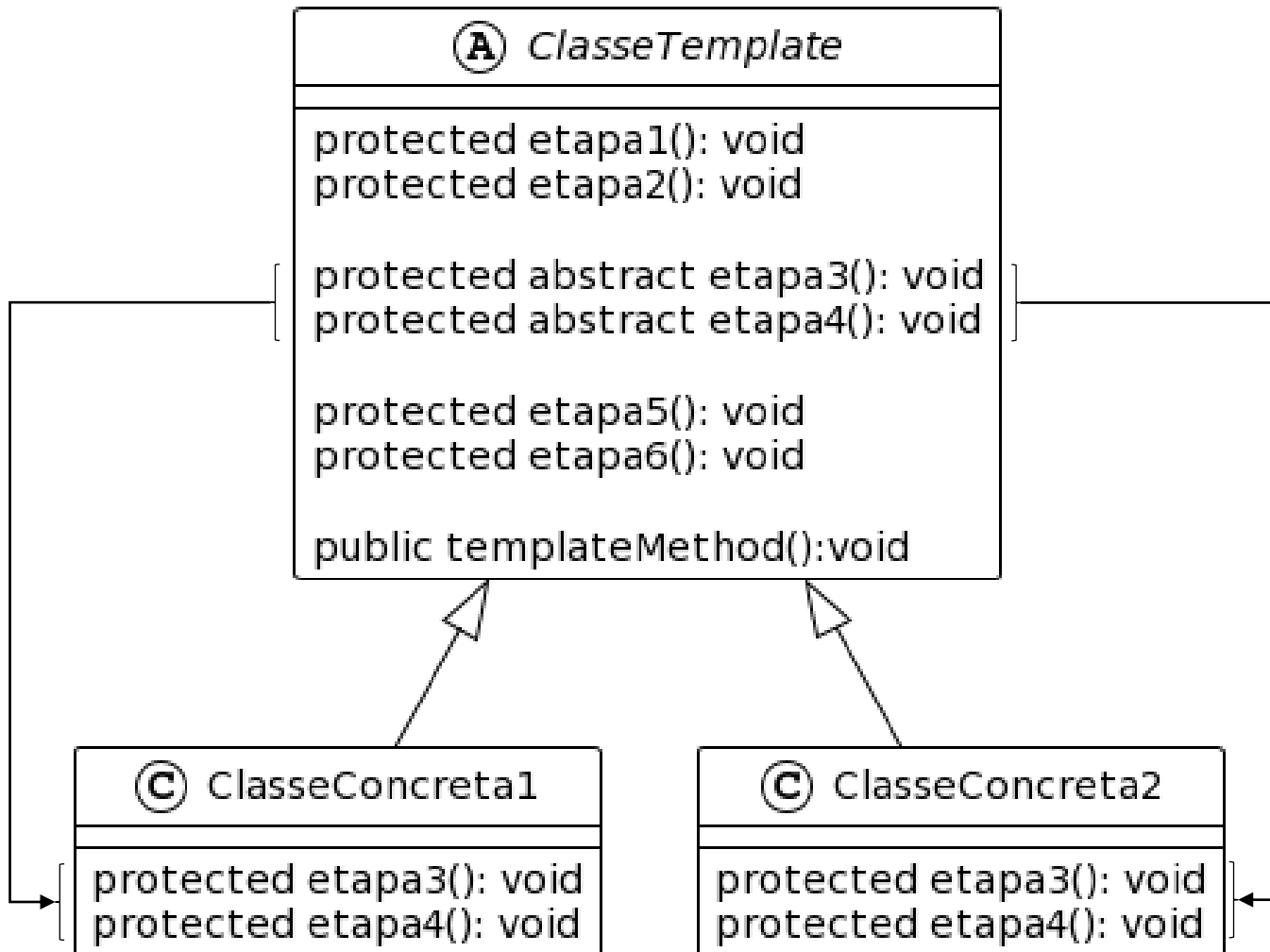
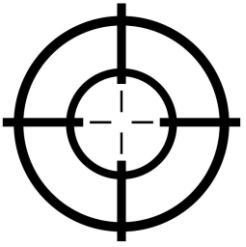
Template method



O método modelo (template method) é um padrão de projeto comportamental que define a estrutura, o esqueleto, de um algoritmo em uma superclasse, mas permite que as subclasses substituam etapas específicas do algoritmo sem alterar sua estrutura.

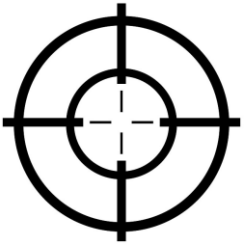
Este padrão auxilia na definição de um algoritmo onde algumas partes são definidas por métodos abstratos. As subclasses devem se responsabilizar por estas partes e fornecer a implementação necessária, ou seja, cada subclasse irá implementar à sua necessidade e oferecer um comportamento concreto para compor o algoritmo final.

Template method UML



A classe abstrata implementa uma série de métodos, que devem ser executados em uma ordem específica, mas alguns métodos permanecem sem implementação, abstratos, o que permite a sobrescrita nas subclasses.

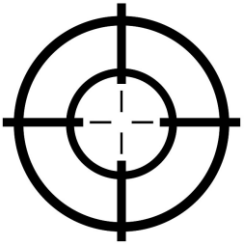
Implementação template method



```
export default abstract class ClasseTemplate {  
  [protected etapa1(): void { ...  
  }  
  [protected etapa2(): void { ...  
  }  
  
  [protected abstract etapa3(): void  
  protected abstract etapa4(): void]  
  
  [protected etapa5(): void { ...  
  }  
  [protected etapa6(): void { ...  
  }  
  
  public templateMethod():void{  
    [this.etapa1();  
    this.etapa2();  
    [this.etapa3();  
    this.etapa4();]  
    [this.etapa5();  
    this.etapa6();]  
  }  
}
```

O template method executa uma série de etapas, mas permite que algumas etapas sejam definidas depois, por subclasses.

Implementação template method

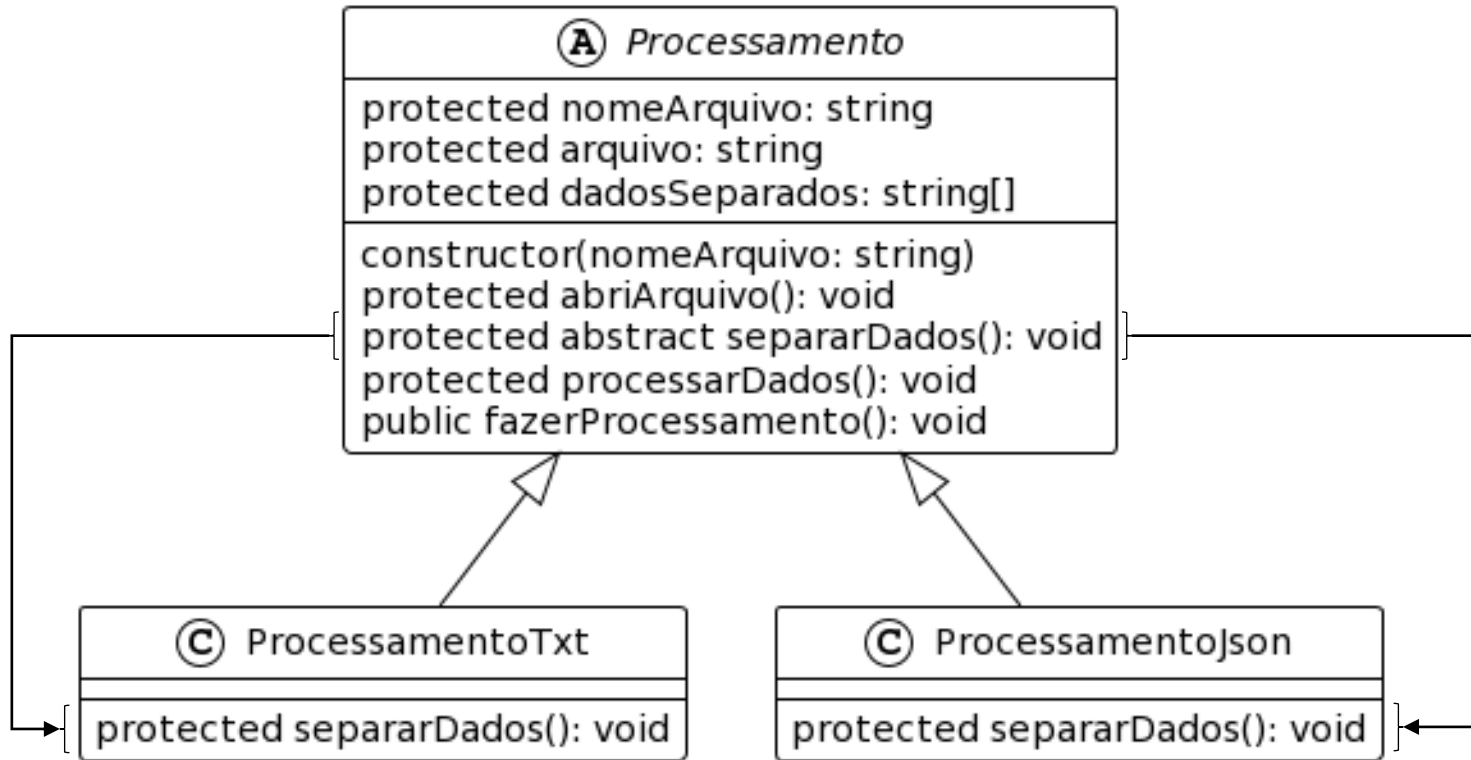
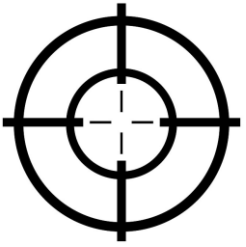


```
export default class ClasseConcreta1 extends ClasseTemplate{  
  protected etapa3(): void {  
    console.log('Execução da etapa 3. Implementação realizada pela ClasseConcreta 1')  
  }  
  protected etapa4(): void {  
    console.log('Execução da etapa 4. Implementação realizada pela ClasseConcreta 1')  
  }  
}
```

```
export default class ClasseConcreta2 extends ClasseTemplate{  
  protected etapa3(): void {  
    console.log('Execução da etapa 3. Implementação realizada pela ClasseConcreta 2')  
  }  
  protected etapa4(): void {  
    console.log('Execução da etapa 4. Implementação realizada pela ClasseConcreta 2')  
  }  
}
```

Cada subclasse deve implementar os métodos de acordo com sua necessidade.

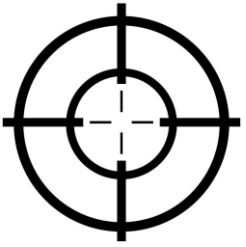
Implementação template method



Exemplo clássico da aplicação do padrão de projeto denominado template method é a abertura e leitura de arquivos.

Neste exemplo a classe **Processamento** detém a lógica para abrir um arquivo. Contudo, dado que o formato do arquivo pode variar, a parte que lê o conteúdo e interpreta suas informações é abstraída para classes especializadas, que são subclasses de **Processamento**.

Implementação template method



A implementação do método que separa os dados, ou seja, lê e retira a informação útil e correta do arquivo é atribuída a subclasse.

```
export default abstract class Processamento {  
  protected nomeArquivo: string  
  protected arquivo!: string  
  protected dadosSeparados!: string[]  
  constructor(nomeArquivo: string) { ...  
  }  
  protected abriArquivo(): void { ...  
  }  
  [protected abstract separarDados(): void  
  protected processarDados(): void { ...  
  }  
  public fazerProcessamento(): void {  
    this.abriArquivo()  
    this.separarDados()  
    this.processarDados()  
  }  
}
```

```
export default class ProcessamentoJson extends Processamento {  
  protected separarDados(): void {  
    let dados = JSON.parse(this.arquivo)  
    let nome = dados['nomeProduto']  
    let valor = dados['valorProduto']  
    let desconto = dados['valorDesconto']  
    let quantidadeParcelas = dados['quantidadeParcelas']  
    this.dadosSeparados = [nome, valor, desconto, quantidadeParcelas]  
  }  
}
```

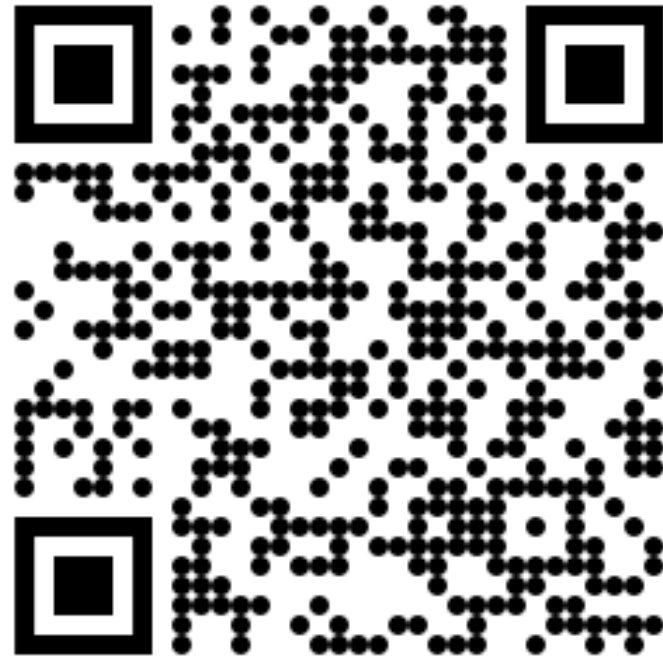

Implementação template method 🦾

<https://github.com/gerson-pn>

[/template-method-typeScript](#)



[/template-method-typeScript-example](#)



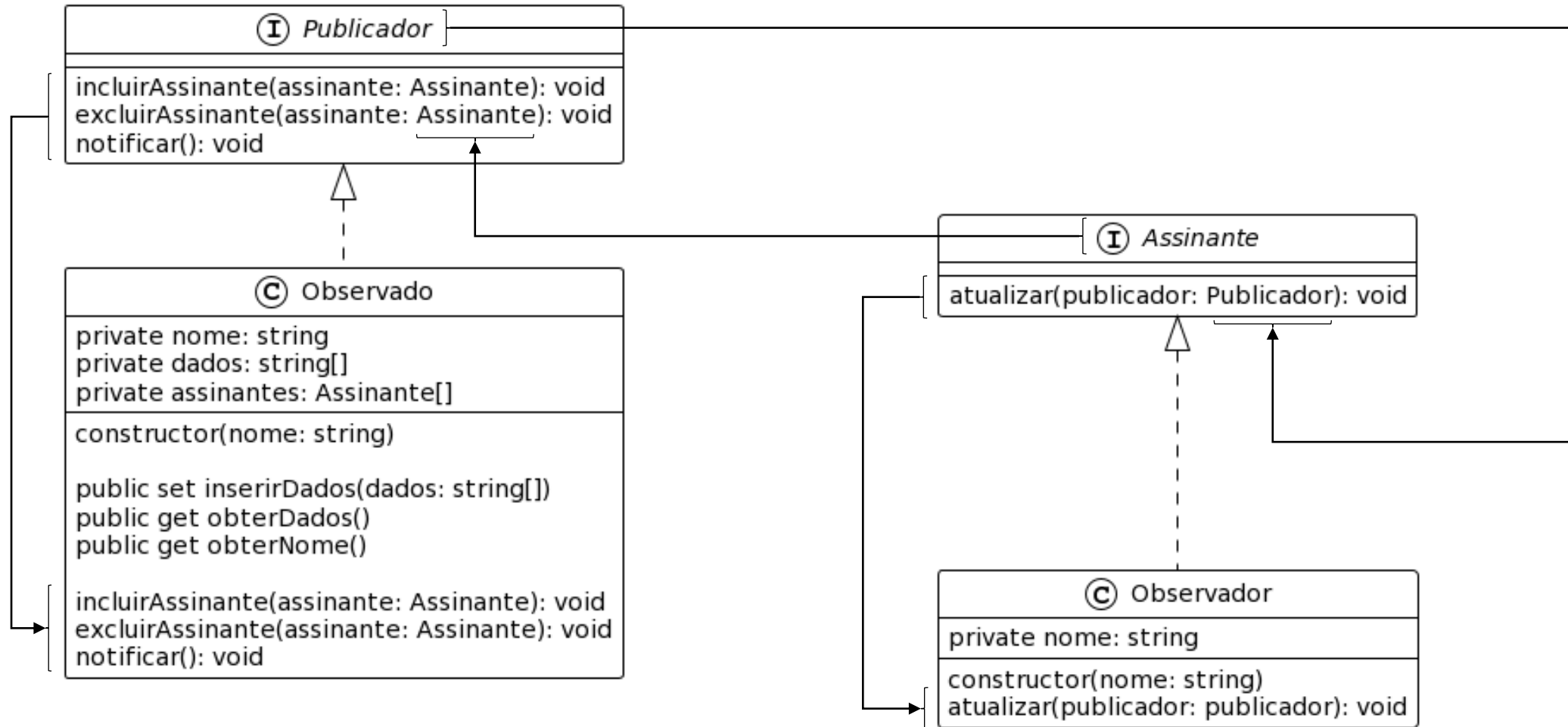
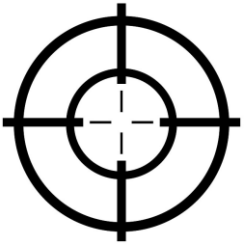
Observer



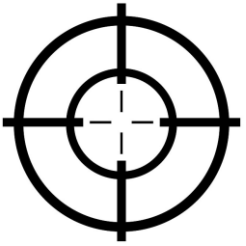
O observador é um padrão de projeto que define uma dependência do tipo “um para muitos” entre objetos, de modo que quando um objeto muda seu estado, todos seus dependentes são notificados e atualizados sobre a mudança, automaticamente.

O padrão implementa um mecanismo de assinatura para notificar múltiplos objetos sobre quaisquer eventos que aconteçam com o objeto que eles estão observando.

Observer UML



Implementação observer

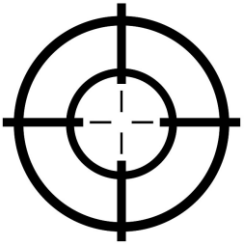


```
export default interface Publicador {  
  incluirAssinante(assinante: Assinante): void  
  excluirAssinante(assinante: Assinante): void  
  notificar(): void  
}
```

```
export default interface Assinante {  
  atualizar(publicador: Publicador): void  
}
```

A interface Publicador defini métodos para inserir e excluir assinantes. Estes métodos devem controlar a inserção ou remoção de um assinante da lista de assinantes.

Implementação observer

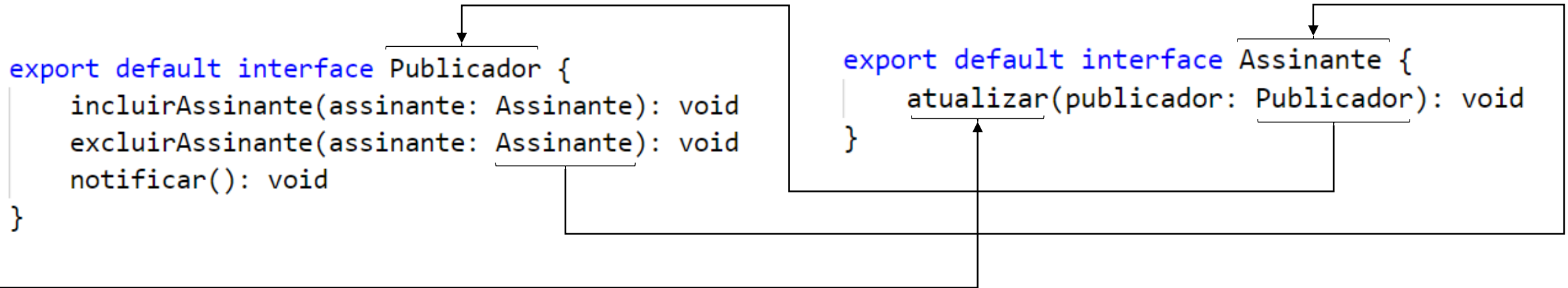
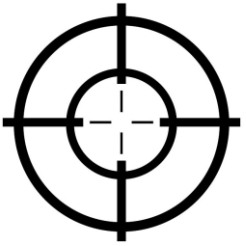


```
export default interface Publicador {  
    incluirAssinante(assinante: Assinante): void  
    excluirAssinante(assinante: Assinante): void  
    notificar(): void  
}
```

```
export default interface Assinante {  
    atualizar(publicador: Publicador): void  
}
```

A interface Publicador defini o método notificar. Este método deve ser disparado sempre que for necessário notificar os assinantes.

Implementação observer



A interface `Assinante` defini o método `atualizar`. Este método será disparado sempre que um assinante for notificado de alguma alteração no `Publicador`. O método `notificar` pode receber vários parâmetros, mas, geralmente, as implementações do padrão passam o valor do objeto `Publicador`, para que seus valores sejam lidos dentro do método.

```
export default class Observado implements Publicador {
```

```
  private nome: string
```

```
  private dados!: string[]
```

```
  private assinantes: Assinante[] = []
```

```
  constructor(nome: string){ ...
```

```
  }
```

```
  public get obterDados(){ ...
```

```
  }
```

```
  public get obterNome(){ ...
```

```
  }
```

```
  public set inserirDados(dados: string[]) {
```

```
    this.dados = dados
```

```
    this.notificar() ←
```

```
  }
```

```
  incluirAssinante(assinante: Assinante): void {
```

```
    this.assinantes.push(assinante)
```

```
  }
```

```
  excluirAssinante(assinante: Assinante): void {
```

```
    let indice = this.assinantes.indexOf(assinante)
```

```
    this.assinantes.splice(indice, 1)
```

```
  }
```

```
  notificar(): void {
```

```
    for (const assinante of this.assinantes) {
```

```
      assinante.atualizar(this)
```

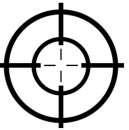
```
    }
```

```
  }
```

```
}
```

Interface que define os métodos que todo Publicador deve implementar.

Métodos definidos pela interface e implementados na classe.

Implementação observer 

```
export default class Observador implements Assinante {  
  private nome: string  
  
  constructor(nome: string) {  
    this.nome = nome  
  }  
  atualizar(publicador: publicador): void {  
    let objeto = publicador as Observado  
    console.log(`-----`)  
    console.log(`Sou o observador ${this.nome} e recebi uma atualização do objeto ${objeto.obterNome}`)  
    console.log(`Os dados modificados foram: ${objeto.obterDados}`)  
  }  
}
```

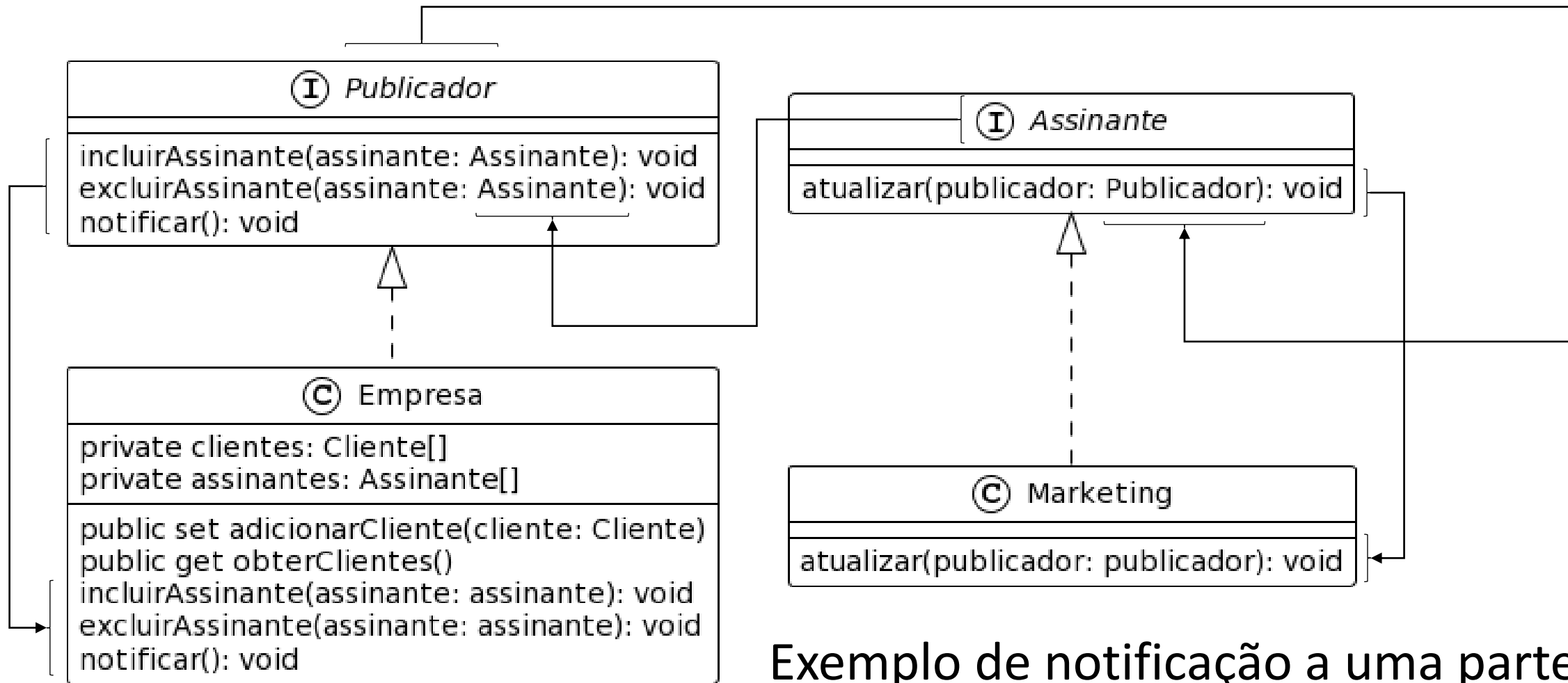
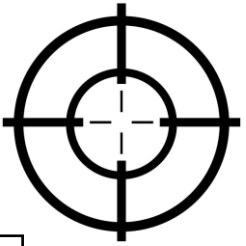
Interface que define o método que todo Assinante deve implementar.

O método atualizar, definido pela interface Assinante e implementado na classe, recebe o próprio Publicador com parâmetro.

Sempre que a classe Publicador notificar seus assinantes, para cada assinante será disparado o método atualizar. É neste método que o assinante define o que lhe é conveniente fazer com o publicador que disparou a atualização.

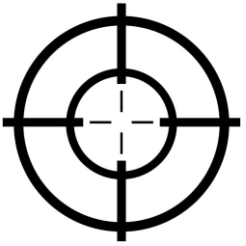
Implementação observer 

Implementação observer



Exemplo de notificação a uma parte específica de um sistema ou aplicação.

Implementação observer

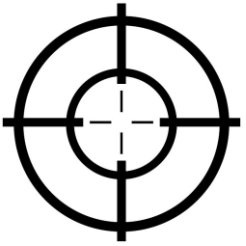


```
export default class Empresa implements Publicador {  
  private clientes: Cliente[] = []  
  private assinantes: Assinante[] = []  
  
  public set adicionarCliente(cliente: Cliente) {  
    this.clientes.push(cliente)  
    this.notificar()  
  }  
  public get obterClientes() {  
    return this.clientes  
  }  
  incluirAssinante(assinante: assinante): void { ...  
  }  
  excluirAssinante(assinante: assinante): void { ...  
  }  
  notificar(): void { ...  
  }  
}
```

```
export default interface Publicador {  
  incluirAssinante(assinante: Assinante): void  
  excluirAssinante(assinante: Assinante): void  
  notificar(): void  
}
```

A classe Empresa implementa a interface Publicador, isto permite a ela adicionar assinantes, que serão notificados cada vez que um novo cliente for incluído na lista de clientes da empresa.

Implementação observer



```
export default interface Assinante {  
  atualizar(publicador: Publicador): void  
}  
  
export default class Marketing implements Assinante {  
  atualizar(publicador: publicador): void {  
    let empresa = publicador as Empresa  
    let clientes = empresa.obterClientes  
    let cliente = clientes[clientes.length - 1]  
    console.log(`O cliente ${cliente.nome} foi adicionado na base de dados da empresa`)  
    console.log(`Iniciar o atendimento de vendas para o cliente ${cliente.nome}`)  
  }  
}
```

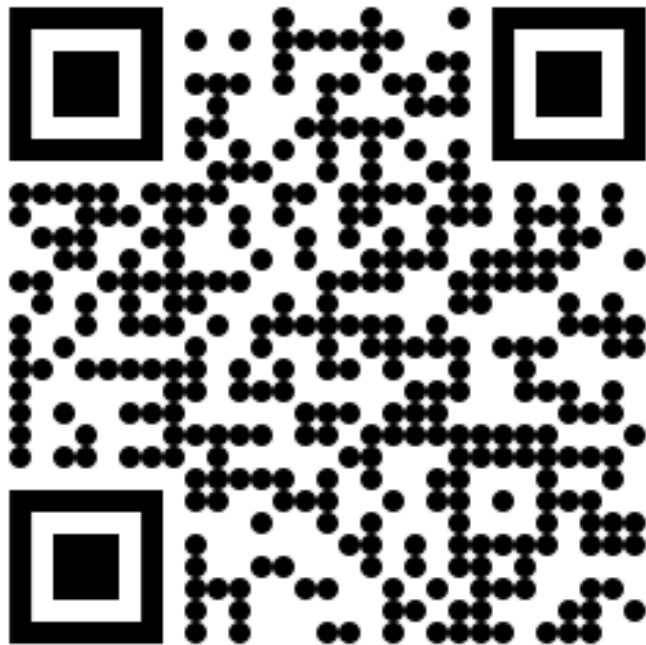
A diagram with a line connecting the 'Assinante' interface to the 'Marketing' class, indicating that the class implements the interface. There are also vertical lines and curly braces on the left side of the code blocks to indicate their structure.

A classe Marketing implementa a interface Assinante, isto permite a ela ser notificada por um Publicador e executar algo com as informações passadas pelo parâmetro do método atualizar.

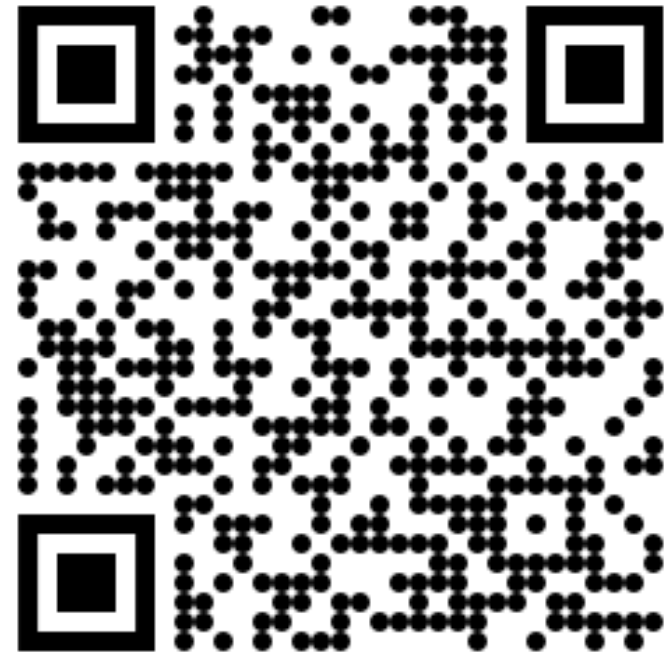
Implementação observer 🦾

<https://github.com/gerson-pn>

/observer-typeScript



/observer-typeScript-example

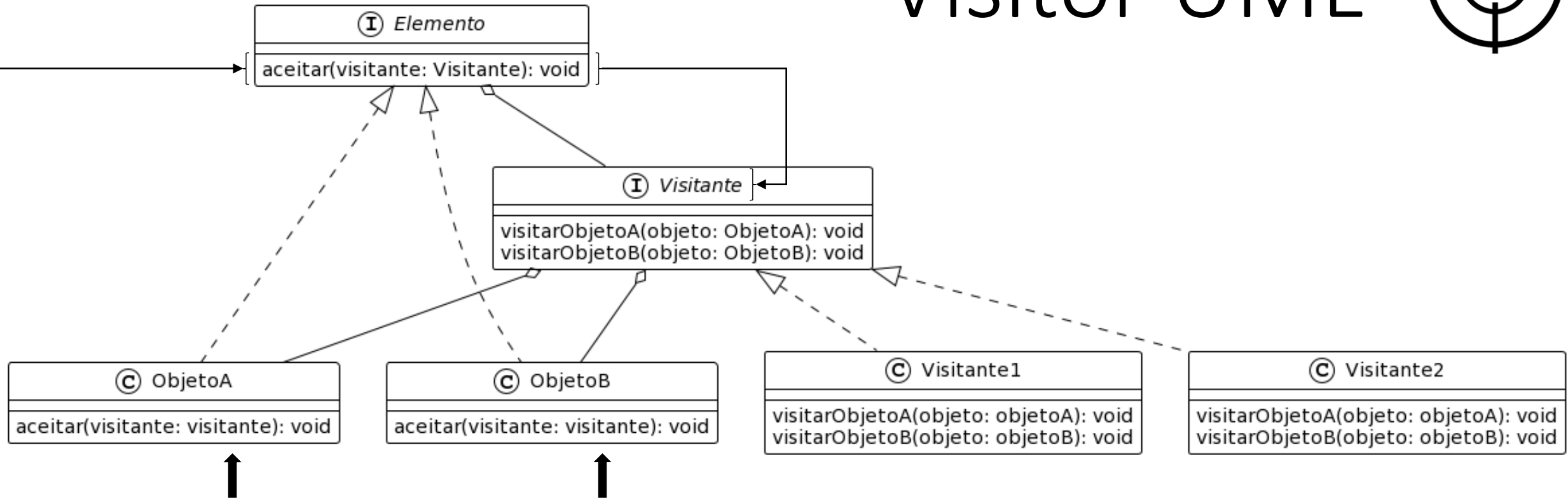
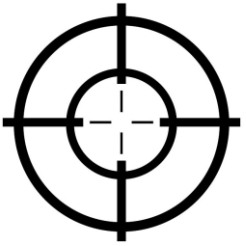


Visitor

É um padrão de projeto comportamental que permite separar algoritmos dos objetos nos quais eles operam. O visitante (visitor) permite que se crie uma nova operação sobre um objeto com pouca ou nenhuma mudança na classe do objeto sobre a qual ele opera.

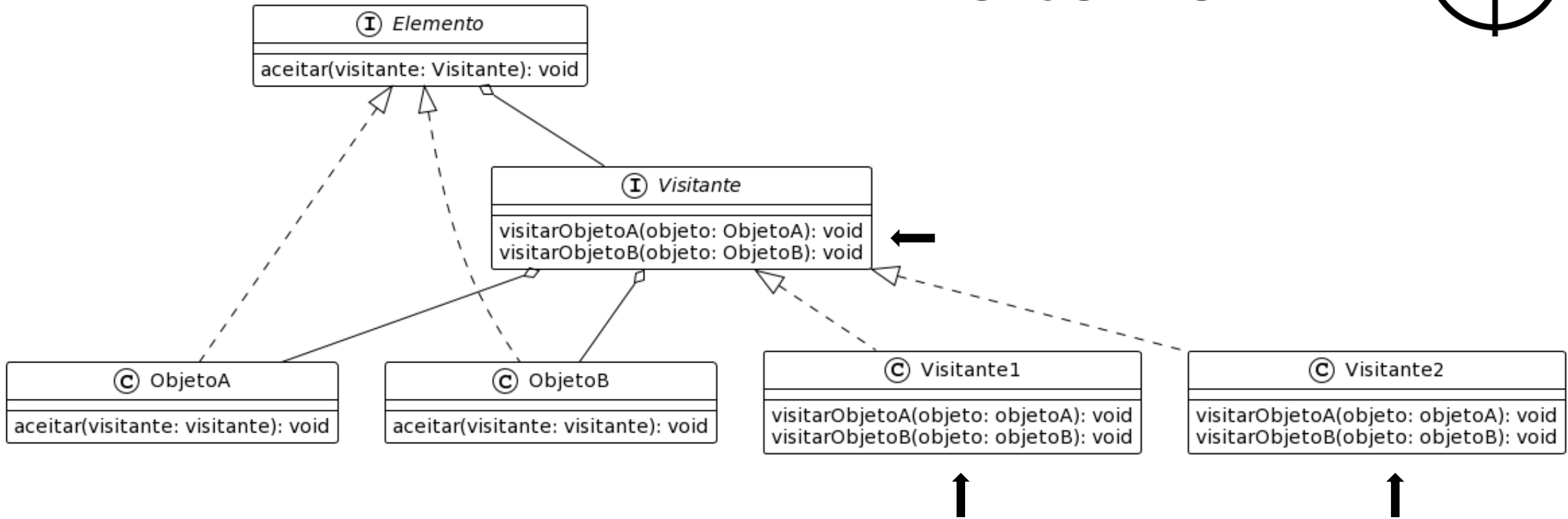
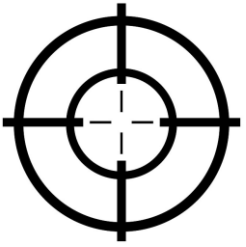
O objetivo principal do padrão visitante é abstrair uma funcionalidade que pode ser aplicada a uma hierarquia agregada de objetos, sem a necessidade de modificar demais os códigos das classes desses objetos.

Visitor UML



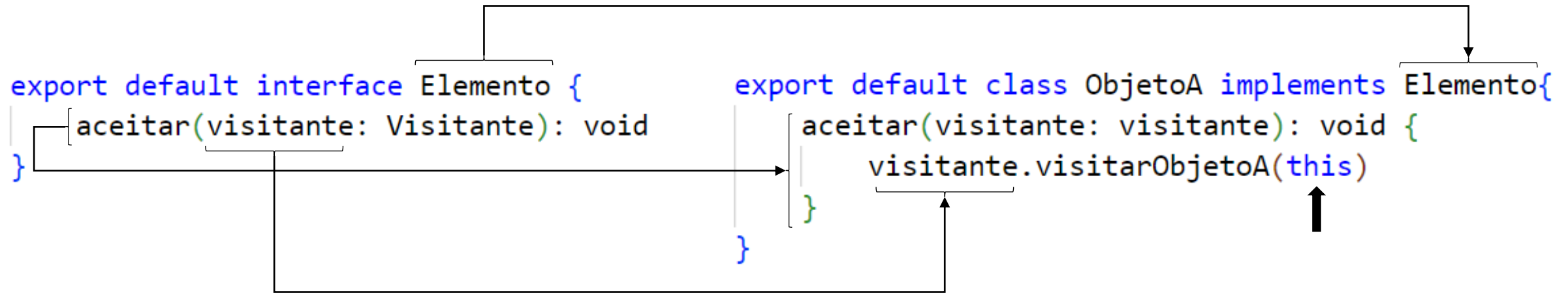
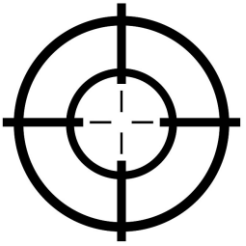
Todo objeto, que pertence a mesma hierarquia agregada, deve implementar uma interface, onde é define-se o método para aceitar um visitante.

Visitor UML



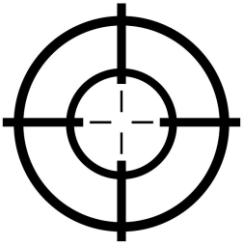
Na interface visitante define-se os métodos para visitar (processar) cada tipo de classe que pertence a hierarquia agregada e, portanto, suas classes concretas sabem com processar cada um dos tipos.

Implementação visitor



A interface Elemento define o método aceitar, que recebe com parâmetro um Visitante. Na classe concreta o método é implementado de modo que a própria classe escolhe qual será o método do Visitante que irá executar sobre ela.

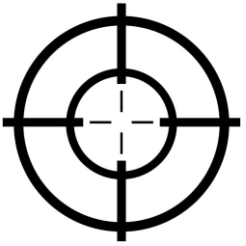
Implementação visitor



```
export default interface Visitante {  
  visitarObjetoA(objeto: ObjetoA): void  
  visitarObjetoB(objeto: ObjetoB): void  
}  
  
export default class Visitante1 implements Visitante {  
  visitarObjetoA(objeto: ObjetoA): void {  
    console.log(`Visitante 1`);  
    console.log(`Processando objeto do tipo A`);  
  }  
  visitarObjetoB(objeto: ObjetoB): void {  
    console.log(`Visitante 1`);  
    console.log(`Processando objeto do tipo B`);  
  }  
}
```

Cada método, definido pela interface Visitante, processa sobre um tipo de objeto.

Implementação visitor



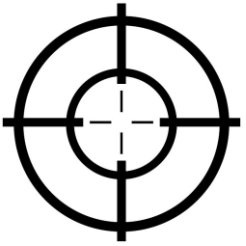
```
export default interface Visitante {  
  visitarObjetoA(objeto: ObjetoA): void  
  visitarObjetoB(objeto: ObjetoB): void  
}
```

```
export default interface Visitante {  
  visitar(objeto: ObjetoA): void  
  visitar(objeto: ObjetoB): void  
}
```

```
export default class Visitante2 implements Visitante{  
  visitar(objeto: ObjetoA): void;  
  visitar(objeto: ObjetoB): void;  
  visitar(objeto: unknown): void {  
    console.log(`Visitante 2`);  
    if (objeto instanceof ObjetoA) {  
      console.log(`Processando o tipo A`)  
    } else if (objeto instanceof ObjetoB) {  
      console.log(`Processando o tipo B`)  
    } else {  
      throw new Error("Sem implementação para o tipo de objeto.")  
    }  
  }  
}
```

Pode-se utilizar a sobrecarga de métodos para definir os métodos que irão processar cada tipo de classe, na interface Visitante.

Implementação visitor



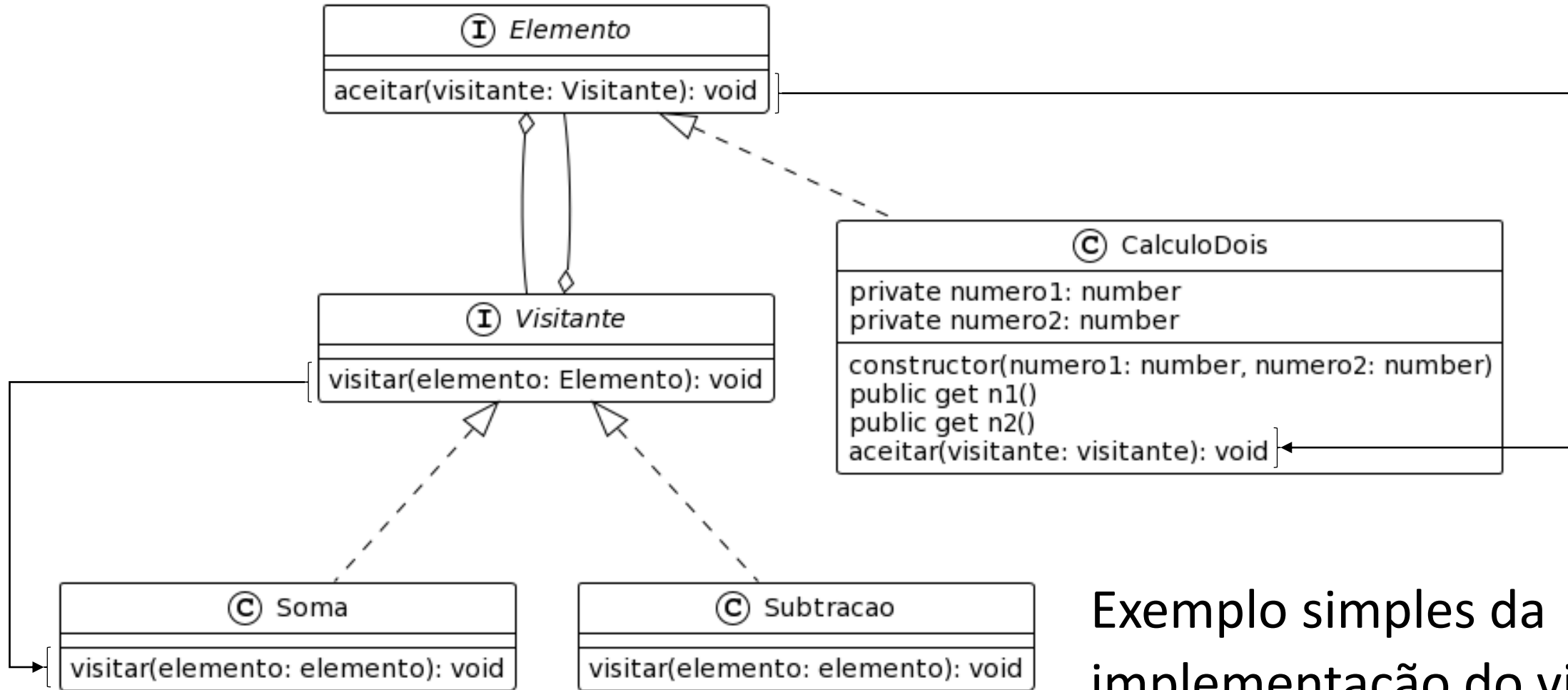
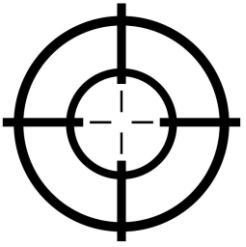
```
export default interface Visitante {  
  visitar(objeto: ObjetoA): void  
  visitar(objeto: ObjetoB): void  
}
```

```
export default interface Visitante {  
  visitar(objeto: Elemento): void  
}
```

```
export default class Visitante1 implements Visitante {  
  visitar(objeto: Elemento): void {  
    console.log(`Visitante 1`);  
    if (objeto instanceof ObjetoA) {  
      console.log(`Processando o tipo A`);  
    } else if (objeto instanceof ObjetoB) {  
      console.log(`Processando o tipo B`);  
    } else {  
      throw new Error("Sem implementação para o tipo de objeto.");  
    }  
  }  
}
```

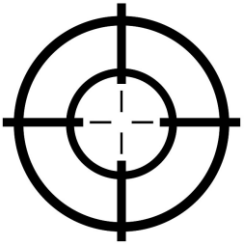
Pode-se utilizar a sobrecarga de métodos para receber apenas um parâmetro, do tipo da interface Elemento.

Implementação visitor



Exemplo simples da
implementação do visitor.

Implementação visitor



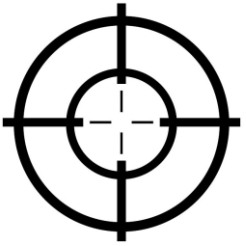
```
export default interface Visitante {  
  visitar(elemento: Elemento): void  
}
```

```
export default interface Elemento {  
  aceitar(visitante: Visitante): void  
}
```

```
export default class CalculoDois implements Elemento {  
  private numero1: number  
  private numero2: number  
  constructor(numero1: number, numero2: number) { ...  
  }  
  public get n1(){ ...  
  }  
  public get n2(){ ...  
  }  
  aceitar(visitante: visitante): void {  
    visitante.visitar(this)  
  }  
}
```

A classe CalculoDois implementa a interface Elemento e o método aceitar.

Implementação visitor



```
export default class CalculoDois implements Elemento {  
  private numero1: number  
  private numero2: number  
  constructor(numero1: number, numero2: number) { ...  
  }  
  public get n1(){ ...  
  }  
  public get n2(){ ...  
  }  
  aceitar(visitante: visitante): void {  
    visitante.visitar(this)  
  }  
}
```

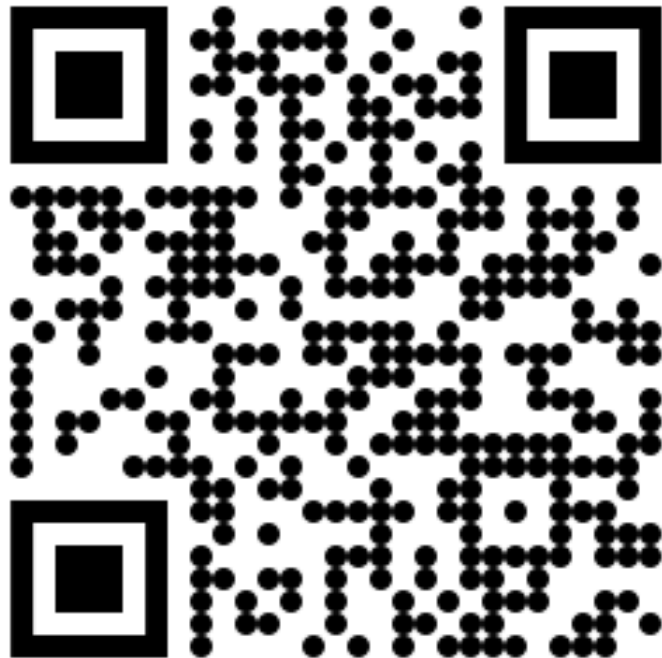
A classe Soma implementa a interface Visitante e o método visitar.

```
export default class Soma implements Visitante{  
  visitar(elemento: elemento): void {  
    let calculo = elemento as CalculoDois  
    let soma = calculo.n1 + calculo.n2  
    console.log(`Resultado da soma: ${soma}`);  
  }  
}
```

Implementação visitor 🦾

<https://github.com/gerson-pn>

/visitor-typeScript



/visitor-typeScript-example



TypeScript

